Deployment types by service - which one supports canary, linear, blue/green, rolling

Quick reference for the most common Domain 1 mix-up: which deployment type each service supports (Lambda, EC2, ECS, EKS, Elastic Beanstalk, API Gateway, CloudFront) and what the setting is called.

Key: YES = built in. **TOOL** = possible only with another tool or by hand. **NO** = not available.

Table 1 - Every deployment type by service

What each deployment type means

Deployment type	What it means
All at once (in place)	Update everything at the same time. Fast, downtime and risk
Rolling	Update in batches. Old and new run together
Rolling with additional batch	Add an extra batch first so capacity never drops
Immutable	A completely new set of instances, then swap
Blue/green	Two full environments, switch all traffic. **Fastest rollback**
Canary	Small % first, wait, then 100%
Linear	Equal steps (+10% every N minutes)
Automatic rollback	Back to the old version on errors

Part A - Lambda, EC2, ECS

Type	Lambda	EC2 / Auto Scaling	ECS
All at once (in place)	**YES** CodeDeploy `LambdaAllAtOnce`, or point the alias to the new version	**YES** CodeDeploy in-place `AllAtOnce`	**YES** CodeDeploy `ECSAllAtOnce`, or rolling with min 0%
Rolling	**NO**	**YES** CodeDeploy in-place `HalfAtATime` / `OneAtATime`, **ASG instance refresh**, CloudFormation `AutoScalingRollingUpdate`	**YES** default (minimumHealthyPercent / maximumPercent)
Rolling with additional batch	**NO**	**TOOL** instance refresh with max healthy above 100%	**YES** maximumPercent above 100 (for example 100/200)
Immutable	**NO** (versions are already immutable)	**YES** CloudFormation `AutoScalingReplacingUpdate`	**NO**
Blue/green	**YES** move the **alias** to the new version	**YES** CodeDeploy **blue/green** (new instances behind the load balancer)	**YES CodeDeploy** blue/green (two target groups)
Canary	**YES** `LambdaCanary10Percent5Minutes` (also 10, 15, 30), via SAM **DeploymentPreference**	**TOOL not in CodeDeploy for EC2**: ALB weighted target groups or Route 53 weighted records	**YES** `ECSCanary10Percent5Minutes` / `15Minutes` (needs **ALB**)
Linear	**YES** `LambdaLinear10PercentEvery1Minute` (also 2, 3, 10)	**NO** in CodeDeploy for EC2	**YES** `ECSLinear10PercentEvery1Minutes` / `3Minutes` (ALB)
Automatic rollback	**YES** CloudWatch alarms + PreTraffic / PostTraffic hooks	**YES** CodeDeploy alarms, instance refresh alarms	**YES circuit breaker** + alarms (rolling); CodeDeploy alarms (blue/green)
Tool used	**SAM / CodeDeploy**	**CodeDeploy**, ASG, CloudFormation	**ECS** itself or **CodeDeploy**

Part B - EKS, Elastic Beanstalk, API Gateway

Type	EKS	Elastic Beanstalk	API Gateway
All at once (in place)	**YES** `Recreate` strategy	**YES All at once** policy	**YES** deploy straight to the stage
Rolling	**YES** default (maxSurge / maxUnavailable)	**YES Rolling** policy	**NO**
Rolling with additional batch	**YES** maxSurge above 0 with maxUnavailable 0	**YES Rolling with additional batch** policy	**NO**
Immutable	**NO**	**YES Immutable** policy	**NO**
Blue/green	**TOOL** switch the Service selector, or new cluster + Route 53	**YES Swap environment URLs**	**TOOL** switch the stage or the custom domain mapping
Canary	**TOOL Argo Rollouts / Flagger**, or ALB Ingress weights	**YES Traffic splitting** (its name for canary)	**YES stage canary release** (% of traffic)
Linear	**TOOL** Argo Rollouts steps	**NO**	**TOOL** raise the canary % by hand
Automatic rollback	**TOOL** Argo Rollouts analysis, `helm --atomic`	**YES** immutable / traffic splitting roll back on failed health	**NO** (manual)
Tool used	kubectl, Helm, **Argo** (**no CodeDeploy**)	**Beanstalk** itself	**API Gateway** itself

Table 2 - Each service at a glance

Service	Deployment types available	Types it does NOT have	Remember it as
Lambda	All at once, **canary**, **linear**, blue/green (alias switch)	Rolling, immutable	Lambda **shifts traffic** with aliases. No servers, so no rolling
EC2 (CodeDeploy)	**In-place** (AllAtOnce, HalfAtATime, OneAtATime) and **blue/green**	**Canary and linear** (CodeDeploy offers them only for Lambda and ECS)	EC2 = **in-place or blue/green only**
EC2 (Auto Scaling)	Rolling (**instance refresh**), immutable (`ReplacingUpdate`)	-	New AMI = **instance refresh**
ECS	Rolling (default), all at once, **blue/green, canary, linear** (CodeDeploy)	Immutable	ECS = **rolling** by itself, the rest through **CodeDeploy**
EKS	Rolling (default), recreate; blue/green and canary **only with other tools**	**CodeDeploy (none)**	EKS = Kubernetes tools. **No CodeDeploy, ever**
Elastic Beanstalk	All at once, rolling, rolling with additional batch, **immutable**, **traffic splitting**, blue/green (**swap URLs**)	Linear	**5 policies + swap URLs**. Canary is called **traffic splitting**
API Gateway	All at once (deploy to stage), **canary** (stage canary)	Rolling, linear, immutable	API Gateway = **stage canary**
CloudFront (bonus)	**Continuous deployment** (a % of traffic, or requests with a header, to a staging distribution)	Rolling, linear	CloudFront = **staging distribution**

Table 3 - Canary and linear names (most confused)

Service	Canary names	Linear names
Lambda	`LambdaCanary10Percent5Minutes`, `10Minutes`, `15Minutes`, `30Minutes`	`LambdaLinear10PercentEvery1Minute`, `2Minutes`, `3Minutes`, `10Minutes`
ECS	`ECSCanary10Percent5Minutes`, `ECSCanary10Percent15Minutes`	`ECSLinear10PercentEvery1Minutes`, `ECSLinear10PercentEvery3Minutes`
EC2	**NO** none	**NO** none
Elastic Beanstalk	Traffic splitting: `NewVersionPercent` + `EvaluationTime`	**NO** none
API Gateway	Stage canary: `percentTraffic`	**NO** (raise the % by hand)

- **Canary** = **two steps**: 10%, wait, then 100%.
- **Linear** = **many equal steps**: 10%, 20%, 30% ... 100%, a fixed number of minutes apart.

Table 4 - Requirement word to answer

If the question says...	Service	Answer
10% of traffic for 5 minutes, then the rest, automatic rollback	Lambda	SAM `DeploymentPreference: Canary10Percent5Minutes` + Alarms
Move traffic in equal steps every minute	Lambda	`Linear10PercentEvery1Minute`
Fastest rollback	Lambda	**Point the alias back** to the old version
Deploy to EC2 a few servers at a time	EC2	CodeDeploy in-place `OneAtATime` / `HalfAtATime`
Deploy to new EC2 instances, then switch the load balancer	EC2	CodeDeploy **blue/green**
Canary deployment to EC2 with CodeDeploy	EC2	**Trap.** Use blue/green, or ALB / Route 53 weights
Replace instances with a new AMI safely	EC2 / ASG	**Instance refresh**
Rolling deploy on ECS with automatic rollback	ECS	**Circuit breaker** (+ alarms)
Test on another port, then 10% canary on ECS	ECS	CodeDeploy blue/green, test listener, `ECSCanary10Percent5Minutes`
Canary on EKS with metric checks	EKS	**Argo Rollouts / Flagger**
CodeDeploy for EKS	EKS	**Trap.** Not supported
Beanstalk full capacity, lowest cost	Beanstalk	**Rolling with additional batch**
Beanstalk failure must not touch running instances	Beanstalk	**Immutable**
Beanstalk 10% of users for 15 minutes	Beanstalk	**Traffic splitting**
Beanstalk separate environment, instant switch	Beanstalk	**Swap environment URLs**
Send 10% of API calls to the new stage version	API Gateway	**Stage canary**
Send 5% of website traffic to new CloudFront settings	CloudFront	**Continuous deployment**

Five rules to stop the mistakes

1. **CodeDeploy canary and linear exist only for Lambda and ECS.** For EC2, CodeDeploy offers only **in-place** or **blue/green**.
2. **EKS never uses CodeDeploy.** Use Kubernetes rolling updates, or **Argo Rollouts / Flagger**.
3. **Beanstalk's canary is called traffic splitting**, and its blue/green is **swap URLs**. It has **no linear** option.
4. **Lambda and API Gateway have no servers, so no rolling updates.** They **shift traffic**: Lambda with aliases, API Gateway with a stage canary.
5. **Rolling = servers or containers** (EC2, ECS, EKS, Beanstalk). **Traffic shifting = canary or linear** (Lambda, ECS, API Gateway, Beanstalk traffic splitting).